

Pythonia

Manual de ruta de especialización

Videojuegos

Ruta de especialización

Aprende programación construyendo mundos que se mueven.



6 módulos | 12 lecciones | 12 semanas en paralelo al tronco

Repasa, practica y registra cada ejecución en tu Cuadernillo-Diario.

Antes de empezar

Crear juegos 2D completos con pygame: bucle de juego, sprites, animación, colisiones, física, estados, niveles, rendimiento y publicación.

Esta ruta crece junto al tronco común. Cada módulo se desbloquea cuando completas sus lecciones requisito del tronco, así nunca llegas a la especialidad con la base floja. En la app, el módulo aparece en el nivel indicado del árbol; si ya cumples los requisitos antes, puedes adelantarlo.

Todo el código de este manual fue ejecutado: la salida que ves es la real. Cada lección incluye una **práctica evaluable** centrada en la lógica de la especialidad, sin librerías externas, para que puedas corregirla en la app. Cada módulo cierra con una **misión** y la ruta termina con un **proyecto final** (el jefe de la temporada).

Tu plan

Módulo	Aparece en	Requisitos del tronco
1. El bucle de juego La ventana y el bucle Dibujar y mover con delta time	Nivel 2	N1.7 Bucles for y while N1.10 Funciones
2. Sprites y animación Sprites y grupos Animación por cuadros	Nivel 2	N2.6 Programación orientada a objetos I N2.7 Programación orientada a objetos II
3. Colisiones Detectar choques Responder a colisiones	Nivel 3	N2.7 Programación orientada a objetos II
4. Física, sonido y eventos Gravedad y salto Eventos propios, oleadas y sonido	Nivel 3	N3.1 Iteradores y generadores
5. Estados y niveles Máquina de estados Niveles desde datos	Nivel 4	N2.4 Archivos, CSV y JSON N4.7 Patrones de diseño pythónicos
6. Rendimiento y publicación Medir y optimizar el cuadro Empaquetar y publicar	Nivel 4	N4.5 Profiling y optimización N3.10 Empaquetado y distribución
Proyecto final: Plataformas 2D con tres niveles	Al cerrar	Todos los módulos

Ritmo sugerido: 30 a 45 minutos al día, 5 días por semana. Lo importante es no cortar la racha.

Módulo 1

El bucle de juego

Requisitos: N1.7 Bucles for y while, N1.10 Funciones. Aparece en el Nivel 2 del árbol.

1 La ventana y el bucle

Todo juego es un bucle que se repite muchas veces por segundo: leer eventos, actualizar el mundo, dibujar y mostrar.

`pygame.display.set_mode` crea la ventana y `pygame.time.Clock().tick(60)` limita el juego a 60 cuadros por segundo para que corra igual en cualquier equipo.

El evento `QUIT` llega cuando el jugador cierra la ventana: siempre debes atenderlo.

Ejemplo

```

1  import pygame
2
3  pygame.init()
4  pantalla = pygame.display.set_mode((640, 360))
5  pygame.display.set_caption("Mi primer juego")
6  reloj = pygame.time.Clock()
7
8  frames, corriendo = 0, True
9  while corriendo:
10     for evento in pygame.event.get():
11         if evento.type == pygame.QUIT:
12             corriendo = False
13     pantalla.fill((14, 59, 67)) # 1) borrar
14     # 2) actualizar 3) dibujar
15     pygame.display.flip()      # 4) mostrar
16     reloj.tick(60)             # máximo 60 FPS
17     frames += 1
18     if frames == 120:          # en tu PC, quita este límite
19         corriendo = False
20
21 print("Frames:", frames, "| tamaño:", pantalla.get_size())
22 pygame.quit()

```

Salida real

```
Frames: 120 | tamaño: (640, 360)
```

Consejos

- Instala con `pip install pygame` dentro de tu entorno virtual.
- Si la ventana no responde, casi siempre es porque no estás leyendo los eventos en cada vuelta.

Ejercicio

Haz que el color de fondo cambie al presionar las teclas R, G o B.

Práctica evaluable**Contador de FPS y presupuesto por cuadro** · La lógica de la lección, sin librerías externas

- `calcular_fps(duraciones_ms)`: recibe cuánto duró cada cuadro en milisegundos y devuelve los FPS promedio redondeados a entero. Sin datos, 0.
- `presupuesto(fps)`: milisegundos disponibles por cuadro, redondeados a 1 decimal.

Plantilla para empezar

```

1 def calcular_fps(duraciones_ms):
2     """[16, 17, 17] -> 60"""
3
4
5 def presupuesto(fps):
6     """presupuesto(60) -> 16.7"""

```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación¿Para qué sirve `reloj.tick(60)`?

- Espera 60 segundos
- Crea 60 eventos
- Limita el juego a 60 cuadros por segundo
- Dibuja 60 objetos

2 Dibujar y mover con delta time

`pygame.draw` dibuja formas; `pygame.Vector2` representa posiciones y velocidades con operaciones vectoriales.

Mueve las cosas en píxeles por segundo multiplicando por `dt` (el tiempo del último cuadro). Así el movimiento es igual aunque los FPS varíen.

Para rebotar basta invertir el componente de la velocidad cuando el objeto toca un borde.

Ejemplo

```

1  import pygame
2
3  pygame.init()
4  pantalla = pygame.display.set_mode((640, 360))
5  pos = pygame.Vector2(100, 180)
6  vel = pygame.Vector2(300, 150) # píxeles por segundo
7  radio = 20
8  dt = 1 / 60                    # en el juego real: dt = reloj.tick(60) / 1000
9
10 for frame in range(1, 181):
11     pos += vel * dt
12     if pos.x - radio < 0 or pos.x + radio > 640:
13         vel.x *= -1
14     if pos.y - radio < 0 or pos.y + radio > 360:
15         vel.y *= -1
16     pantalla.fill("black")
17     pygame.draw.circle(pantalla, "gold", pos, radio)
18     pygame.display.flip()
19     if frame % 60 == 0:
20         print(f"t={frame // 60}s pos=({pos.x:.0f}, {pos.y:.0f}) vel=({vel.x:.0f},
21               ↳ {vel.y:.0f})")
22 pygame.quit()

```

Salida real

```

t=1s pos=(400, 330) vel=(300, 150)
t=2s pos=(550, 205) vel=(-300, -150)
t=3s pos=(250, 55) vel=(-300, -150)

```

Consejos

- Al rebotar, reubica el objeto dentro del borde para evitar que quede 'pegado' fuera.
- `pos.normalize()` y `pos.length()` facilitan movimientos en ocho direcciones.

Ejercicio

Agrega cinco pelotas con velocidades aleatorias que reboten de forma independiente.

Práctica evaluable

Mover y rebotar · La lógica de la lección, sin librerías externas

Escribe `mover_y_rebotar(x, vx, radio, ancho, dt)`: mueve una pelota en el eje X ($x += vx * dt$) y devuelve la tupla (x, vx) . Si el borde izquierdo de la pelota ($x - radio$) queda antes de 0, la reubica en `radio` y la velocidad pasa a ser positiva; si el borde derecho ($x + radio$) pasa de `ancho`, la reubica en `ancho - radio` y la velocidad pasa a ser negativa.

Plantilla para empezar

```

1  def mover_y_rebotar(x, vx, radio, ancho, dt):
2      """Devuelve (x, vx) tras moverse dt segundos."""
3      x += vx * dt
4      return x, vx

```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

¿Por qué se multiplica la velocidad por dt ?

- a) Para dibujar más rápido
- b) Para que el movimiento no dependa de los FPS
- c) Para ahorrar memoria
- d) Es obligatorio en pygame

Misión del módulo

Crea una pelota que rebote por la pantalla y cambie de color en cada choque con un borde.

Módulo 2

Sprites y animación

Requisitos: N2.6 Programación orientada a objetos I, N2.7 Programación orientada a objetos II. Aparece en el Nivel 2 del árbol.

3 Sprites y grupos

Un `pygame.sprite.Sprite` combina una imagen (`image`) y una posición (`rect`). Heredar de él es POO aplicada.

Los `Group` actualizan y dibujan muchos sprites de una sola vez con `update()` y `draw()`.

`rect.clamp_ip` mantiene un rectángulo dentro de otro: perfecto para que el jugador no salga de la pantalla.

Ejemplo

```

1  import pygame
2
3  pygame.init()
4  pygame.display.set_mode((640, 360))
5
6  class Nave(pygame.sprite.Sprite):
7      def __init__(self, x, y):
8          super().__init__()
9          self.image = pygame.Surface((40, 30))
10         self.image.fill("deepskyblue")
11         self.rect = self.image.get_rect(center=(x, y))
12         self.velocidad = 5
13
14     def update(self, teclas):
15         if teclas.get("derecha"):
16             self.rect.x += self.velocidad
17         if teclas.get("izquierda"):
18             self.rect.x -= self.velocidad
19         self.rect.clamp_ip(pygame.Rect(0, 0, 640, 360))
20
21 jugador = Nave(320, 330)
22 todos = pygame.sprite.Group(jugador)
23 for _ in range(10):
24     todos.update({"derecha": True}) # en el juego: pygame.key.get_pressed()
25 print("Tras 10 frames:", jugador.rect.center)
26 for _ in range(100):
27     todos.update({"derecha": True})
28 print("Contra el borde:", jugador.rect.right)
29 pygame.quit()

```

Salida real

```

Tras 10 frames: (370, 330)
Contra el borde: 640

```

Consejos

- Carga imágenes con `pygame.image.load(ruta).convert_alpha()` después de crear la ventana.
- Usa varios grupos (jugadores, enemigos, balas) para controlar qué choca con qué.

Ejercicio

Agrega disparos: al presionar espacio, crea un sprite Bala que suba y se elimine al salir de la pantalla.

Práctica evaluable

Una nave que no se sale de la pantalla · La lógica de la lección, sin librerías externas

Crea la clase `Nave(x, ancho_pantalla=640, ancho=40, velocidad=5)` con el método `mover(direccion):` `direccion` es -1 (izquierda), 0 o 1 (derecha). Mueve la nave `velocidad` píxeles en esa dirección, la mantiene entre 0 y `ancho_pantalla - ancho` y devuelve la nueva `x`.

Plantilla para empezar

```

1 class Nave:
2     def __init__(self, x, ancho_pantalla=640, ancho=40, velocidad=5):
3         self.x = x
4
5     def mover(self, direccion):
6         """-1 izquierda, 0 quieta, 1 derecha. Devuelve la nueva x."""

```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

¿Qué dos atributos necesita un Sprite para que Group.draw() funcione?

- a) surface y pos
- b) color y size
- c) image y rect
- d) x e y

4

Animación por cuadros

Animar es mostrar imágenes en secuencia. El cuadro actual se calcula a partir del tiempo transcurrido, no de los frames, para que la animación tenga su propia velocidad.

Una clase Animacion guarda los cuadros, la duración de cada uno y el tiempo acumulado.

Las propiedades (@property) permiten consultar el índice y la imagen actual como si fueran atributos.

Ejemplo

```

1  import pygame
2
3  pygame.init()
4  pygame.display.set_mode((100, 100))
5
6  cuadros = []
7  for color in ["red", "orange", "yellow", "orange"]:
8      s = pygame.Surface((32, 32))
9      s.fill(color)
10     cuadros.append(s)
11
12  class Animacion:
13      def __init__(self, cuadros, fps):
14          self.cuadros, self.duracion, self.tiempo = cuadros, 1 / fps, 0.0
15      def update(self, dt):
16          self.tiempo += dt
17      @property
18      def indice(self):
19          return int(self.tiempo / self.duracion) % len(self.cuadros)
20      @property
21      def imagen(self):
22          return self.cuadros[self.indice]
23
24  fuego = Animacion(cuadros, fps=8)
25  secuencia = []
26  for _ in range(30):          # 30 frames a 60 FPS = medio segundo
27      fuego.update(1 / 60)
28      secuencia.append(fuego.indice)
29  print(secuencia)
30  print("Color actual:", tuple(fuego.imagen.get_at((0, 0))))
31  pygame.quit()

```

Salida real

```

[0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 3, 3, 3]
Color actual: (255, 165, 0, 255)

```

Consejos

- Las hojas de sprites (sprite sheets) guardan todos los cuadros en una imagen; recórtalos con subsurface.
- Busca recursos gratuitos con licencia clara en kenney.nl u opengameart.org.

Ejercicio

Crea animaciones distintas para 'quieto', 'caminar' y 'saltar', y cambia entre ellas según el estado del jugador.

Práctica evaluable

Elegir el cuadro de una animación · La lógica de la lección, sin librerías externas

Escribe `indice_cuadro(tiempo, fps, cantidad, bucle=True)`: con tiempo en segundos desde que empezó la animación, fps cuadros por segundo de la animación y cantidad de cuadros, devuelve el índice a mostrar. Si bucle es True, vuelve a empezar al terminar; si es False, se queda en el último cuadro.

Plantilla para empezar

```

1 def indice_cuadro(tiempo, fps, cantidad, bucle=True):
2     """indice_cuadro(0.5, 8, 4) -> 0 (4 cuadros: la vuelta completa dura 0,5 s)"""

```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

¿Por qué calcular el cuadro con el tiempo y no con el número de frames?

- a) La animación mantiene su velocidad aunque cambien los FPS
- b) pygame lo exige
- c) Es más corto
- d) Usa menos memoria

Misión del módulo

Crea una nave controlable con animación de propulsor y límites de pantalla.

Módulo 3

Colisiones

Requisitos: N2.7 Programación orientada a objetos II. Aparece en el Nivel 3 del árbol.

5 Detectar choques

`Rect.colliderect` indica si dos rectángulos se superponen. Es rápido y suficiente para la mayoría de los juegos 2D.

`pygame.sprite.spritecollide(sprite, grupo, dokill)` devuelve los sprites del grupo que tocan al sprite; con `dokill=True` los elimina del grupo.

Para formas irregulares existen las máscaras (`pygame.mask`), más precisas pero más costosas.

Ejemplo

```

1 import pygame, random
2
3 pygame.init()
4 pygame.display.set_mode((640, 360))
5 random.seed(7)
6
7 class Bloque(pygame.sprite.Sprite):
8     def __init__(self, x, y, w=20, h=20):
9         super().__init__()
10        self.image = pygame.Surface((w, h))
11        self.rect = self.image.get_rect(topleft=(x, y))
12
13 jugador = Bloque(300, 160, 40, 40)
14 monedas = pygame.sprite.Group(Bloque(random.randint(0, 620), random.randint(0,
15 ← 340)) for _ in range(30))
16 monedas.add(Bloque(310, 170), Bloque(325, 185)) # dos monedas bajo el jugador
17 print("Monedas al inicio:", len(monedas))
18
19 recogidas = pygame.sprite.spritecollide(jugador, monedas, dokill=True)
20 print("Recogidas:", len(recogidas), "| quedan:", len(monedas))
21 print("¿Toca la puerta?", jugador.rect.colliderect(pygame.Rect(338, 198, 10, 10)))
22 pygame.quit()

```

Salida real

```

Monedas al inicio: 32
Recogidas: 2 | quedan: 30
¿Toca la puerta? True

```

Consejos

- Revisa colisiones después de mover y antes de dibujar.
- `pygame.sprite.groupcollide` compara dos grupos completos (balas contra enemigos).

Ejercicio

Suma 10 puntos por moneda, resta una vida al tocar un enemigo y muestra ambos valores en pantalla con `pygame.font`.

Práctica evaluable

Colisiones de rectángulos · La lógica de la lección, sin librerías externas

Los rectángulos son tuplas (x, y, ancho, alto).

- `colisionan(a, b)`: True si se superponen. Si solo se tocan en el borde, no cuenta.
- `recoger(jugador, monedas)`: devuelve (restantes, recogidas), con la lista de monedas que no toca el jugador y cuántas tocó.

Plantilla para empezar

```

1 def colisionan(a, b):
2     """colisionan((0, 0, 10, 10), (5, 5, 10, 10)) -> True"""
3
4
5 def recoger(jugador, monedas):
6     """-> (restantes, recogidas)"""

```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

¿Qué hace dokill=True en spritecollide?

- a) Cierra el juego
- b) Quita del grupo los sprites que chocaron
- c) Detiene la animación
- d) Elimina al jugador

6 Responder a colisiones

Detectar no basta: hay que corregir la posición. En un juego de plataformas se resuelve un eje a la vez: primero se mueve en X y se corrige, luego en Y y se corrige.

Al chocar desde arriba, el jugador queda apoyado (`en_suelo = True`) y su velocidad vertical vuelve a cero.

Resolver por ejes evita que el personaje se 'enganche' en las esquinas.

Ejemplo

```

1  import pygame
2
3  pygame.init()
4  pygame.display.set_mode((640, 360))
5  paredes = [pygame.Rect(0, 300, 640, 60), pygame.Rect(200, 220, 120, 20)] # suelo
6  jugador = pygame.Rect(150, 100, 30, 40)
7  vel = pygame.Vector2(4, 0)
8  GRAVEDAD = 0.8
9
10 def mover(rect, vel):
11     en_suelo = False
12     rect.x += round(vel.x) # 1) eje X
13     for p in paredes:
14         if rect.colliderect(p):
15             if vel.x > 0: rect.right = p.left
16             elif vel.x < 0: rect.left = p.right
17     rect.y += round(vel.y) # 2) eje Y
18     for p in paredes:
19         if rect.colliderect(p):
20             if vel.y > 0:
21                 rect.bottom, vel.y, en_suelo = p.top, 0, True
22             elif vel.y < 0:
23                 rect.top, vel.y = p.bottom, 0
24     return en_suelo
25
26 for frame in range(60):
27     vel.y += GRAVEDAD
28     en_suelo = mover(jugador, vel)
29     if frame in (0, 15, 30, 59):
30         print(f"frame {frame:2}: x={jugador.x:3} y={jugador.y:3} suelo={en_suelo}")
31     pygame.quit()

```

Salida real

```
frame 0: x=154 y=101 suelo=False
frame 15: x=214 y=180 suelo=True
frame 30: x=274 y=180 suelo=True
frame 59: x=390 y=260 suelo=True
```

Consejos

- Si un objeto rápido atraviesa paredes, divide su movimiento en pasos más pequeños.
- Guarda en_suelo para permitir saltar solo cuando el personaje está apoyado.

Ejercicio

Agrega salto con la tecla espacio que solo funcione cuando en_suelo es verdadero.

Práctica evaluable

Aterrizar sobre plataformas · La lógica de la lección, sin librerías externas

Escribe mover_vertical(jugador, vy, plataformas): jugador es (x, y, ancho, alto) y las plataformas también son rectángulos. Mueve al jugador vy píxeles en Y y, si choca con una plataforma, lo corrige: si caía (vy > 0) queda apoyado encima, con vy = 0 y en_suelo = True; si subía (vy < 0) queda justo debajo, con vy = 0. Devuelve (y, vy, en_suelo).

Plantilla para empezar

```
1 def mover_vertical(jugador, vy, plataformas):
2     """-> (y, vy, en_suelo)"""
3     x, y, w, h = jugador
4     y += vy
5     return y, vy, False
```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

¿Por qué resolver las colisiones un eje a la vez?

- pygame no permite diagonales
- Es más rápido de escribir
- Para saber de qué lado vino el choque y corregir sin engancharse
- Para ahorrar memoria

Misión del módulo

Crea un juego de recolectar monedas con obstáculos y puntaje.

Módulo 4

Física, sonido y eventos

Requisitos: N3.1 Iteradores y generadores. Aparece en el Nivel 3 del árbol.

7 Gravedad y salto

La física de un juego no necesita ser real, necesita sentirse bien. La gravedad suma velocidad hacia abajo en cada cuadro; el salto es una velocidad inicial negativa.

Una velocidad terminal (un máximo) evita caídas demasiado rápidas y túneles a través del suelo.

Ajustar gravedad y fuerza de salto cambia por completo la sensación del juego: experimenta.

Ejemplo

```

1 GRAVEDAD, SALTO, VEL_MAX = 0.6, -12, 15
2 suelo = 300.0
3 y, vy = suelo, 0.0
4 alturas = []
5
6 for frame in range(45):
7     if frame == 0:
8         vy = SALTO # el jugador presiona espacio
9         vy = min(vy + GRAVEDAD, VEL_MAX) # velocidad terminal
10        y = min(y + vy, suelo)
11        if y == suelo:
12            vy = 0
13        alturas.append(round(suelo - y))
14
15 print(alturas[:4])
16 print("Altura máxima:", max(alturas), "px")

```

Salida real

```

[11, 51, 81, 101, 112, 113, 105, 87, 59, 22, 0, 0]
Altura máxima: 114 px

```

Consejos

- 'Coyote time': permite saltar unos pocos cuadros después de dejar la plataforma; el juego se siente más justo.
- Salto variable: si el jugador suelta la tecla antes, corta la subida.

Ejercicio

Implementa salto variable: si se suelta espacio mientras sube, multiplica vy por 0.5.

Práctica evaluable**Simular un salto** · La lógica de la lección, sin librerías externas

Escribe `simular_salto(gravedad, fuerza, vel_max, frames)`: el jugador empieza en el suelo (altura 0) y salta con velocidad vertical - fuerza (negativa es hacia arriba). En cada cuadro: la velocidad suma gravedad sin pasar de `vel_max`, la posición suma la velocidad y no puede bajar del suelo (si llega, la velocidad vuelve a 0). Devuelve la lista de alturas (positivas hacia arriba) de cada cuadro.

Plantilla para empezar

```

1 def simular_salto(gravedad, fuerza, vel_max, frames):
2     """simular_salto(1, 4, 10, 6) -> [3, 5, 6, 6, 5, 3]"""
3     y, vy = 0, -fuerza
4     alturas = []
5     return alturas

```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

¿Qué evita la velocidad terminal?

- a) Caídas cada vez más rápidas que atraviesan el suelo
- b) Los rebotes
- c) Que el juego se cierre
- d) Que el jugador salte

8 Eventos propios, oleadas y sonido

`pygame.time.set_timer` genera eventos propios cada cierto tiempo: aparición de enemigos, cuenta regresiva, power-ups.

Un generador es perfecto para describir oleadas infinitas con dificultad creciente: cada `next()` entrega la siguiente.

`pygame.mixer.Sound` reproduce efectos y `pygame.mixer.music` la música de fondo. Inicializa el audio con manejo de errores: no todos los equipos tienen.

Ejemplo

```

1  import pygame
2
3  pygame.init()
4  pygame.display.set_mode((320, 180))
5  NUEVA_OLEADA = pygame.USEREVENT + 1
6  pygame.time.set_timer(NUEVA_OLEADA, 100) # cada 100 ms
7
8  def oleadas():
9      n = 1
10     while True:
11         yield {"numero": n, "enemigos": 3 + 2 * n, "velocidad": round(1 + 0.25 * n,
12         ↪ 2)}
13         n += 1
14
15     try:
16         pygame.mixer.init()
17         print("Audio listo:", pygame.mixer.get_init() is not None)
18     except pygame.error:
19         print("Sin audio: el juego sigue en silencio")
20
21     generador, recibidas, reloj = oleadas(), 0, pygame.time.Clock()
22     while recibidas < 3:
23         for e in pygame.event.get():
24             if e.type == NUEVA_OLEADA:
25                 recibidas += 1
26                 print(next(generador))
27                 reloj.tick(60)
28     pygame.quit()

```

Salida real

```

Audio listo: True
{'numero': 1, 'enemigos': 5, 'velocidad': 1.25}
{'numero': 2, 'enemigos': 7, 'velocidad': 1.5}
{'numero': 3, 'enemigos': 9, 'velocidad': 1.75}

```

Consejos

- Usa .ogg para música y .wav para efectos cortos.
- Precarga los sonidos al inicio: cargarlos durante el juego causa tirones.

Ejercicio

Reproduce un sonido al comenzar cada oleada y sube el volumen de la música a medida que aumenta la dificultad.

Práctica evaluable

Generador de oleadas · La lógica de la lección, sin librerías externas

Escribe el generador infinito `oleadas(base=3, incremento=2)`: cada `next()` entrega un diccionario `{"numero": n, "enemigos": base + incremento * n, "velocidad": 1 + 0.25 * n}` con `n = 1, 2, 3...`

Escribe también `tomar(generador, n)`, que devuelve una lista con los próximos `n` valores.

Plantilla para empezar

```

1 def oleadas(base=3, incremento=2):
2     """Generador infinito de oleadas."""
3     n = 1
4
5
6 def tomar(generator, n):
7     """Los próximos n valores del generador."""

```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

¿Qué hace `pygame.time.set_timer(EVENTO, 100)`?

- a) Envía EVENTO a la cola cada 100 ms
- b) Crea 100 eventos de golpe
- c) Pausa 100 ms
- d) Limita a 100 FPS

Misión del módulo

Crea un juego de oleadas con dificultad creciente, efectos de sonido y música de fondo.

Módulo 5

Estados y niveles

Requisitos: N2.4 Archivos, CSV y JSON, N4.7 Patrones de diseño pythónicos. Aparece en el Nivel 4 del árbol.

9

Máquina de estados

Un juego tiene pantallas o estados: menú, jugando, pausa, game over. Cada estado decide cómo reacciona a los eventos y a cuál pasa después.

El patrón State representa cada estado como una clase con el mismo método `manejar`. El juego solo guarda el estado actual.

Así agregar una pantalla nueva no obliga a tocar un `if` gigante.

Ejemplo

```

1  class Estado:
2      def manejar(self, evento):
3          return self
4      def __str__(self):
5          return type(self).__name__
6
7  class Menu(Estado):
8      def manejar(self, evento):
9          return Jugando() if evento == "enter" else self
10
11 class Jugando(Estado):
12     def __init__(self):
13         self.vidas = 3
14     def manejar(self, evento):
15         if evento == "esc":
16             return Pausa(self)
17         if evento == "golpe":
18             self.vidas -= 1
19             return GameOver() if self.vidas == 0 else self
20         return self
21
22 class Pausa(Estado):
23     def __init__(self, juego):
24         self.juego = juego
25     def manejar(self, evento):
26         return self.juego if evento == "esc" else self
27
28 class GameOver(Estado):
29     def manejar(self, evento):
30         return Menu() if evento == "enter" else self
31
32 estado = Menu()
33 for ev in ["enter", "golpe", "esc", "golpe", "esc", "golpe", "golpe", "enter"]:
34     estado = estado.manejar(ev)
35     extra = f" (vidas={estado.vidas})" if isinstance(estado, Jugando) else ""
36     print(f"{ev:6} -> {estado}{extra}")

```

Salida real

```

enter -> Jugando (vidas=3)
golpe -> Jugando (vidas=2)
esc   -> Pausa
golpe -> Pausa
esc   -> Jugando (vidas=2)
golpe -> Jugando (vidas=1)
golpe -> GameOver
enter -> Menu

```

Consejos

- Fíjate que Pausa guarda la partida en curso: al volver, las vidas siguen igual.
- Agrega métodos `update()` y `draw()` a cada estado para que cada pantalla se dibuje sola.

Ejercicio

Agrega un estado Opciones accesible desde el menú que permita cambiar el volumen.

Práctica evaluable

Máquina de estados con una tabla · La lógica de la lección, sin librerías externas

Otra forma de escribir una máquina de estados es con una tabla de transiciones. Ya existe TRANSICIONES, un diccionario {(estado, evento): nuevo_estado}.

- `transicion(estado, evento)`: devuelve el nuevo estado, o el mismo si esa combinación no está en la tabla.
- `recorrer(eventos, inicial="menu")`: devuelve la lista de estados por los que pasa, empezando por el inicial.

Datos que prepara el evaluador (también prueba con otros valores)

```
1 TRANSICIONES = {"menu", "enter"): "jugando", ("jugando", "esc"): "pausa",
2   ↳ ("pausa", "esc"): "jugando",
   ("jugando", "morir"): "game_over", ("game_over", "enter"): "menu"}
```

Plantilla para empezar

```
1 def transicion(estado, evento):
2     """transicion("menu", "enter") -> "jugando" """
3
4
5 def recorrer(eventos, inicial="menu"):
6     """["enter", "esc"] -> ["menu", "jugando", "pausa"]"""
```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

En el ejemplo, ¿por qué al quitar la pausa se conservan las vidas?

- Se reinician siempre
- Por el GIL
- Se guardan en un archivo
- Pausa guarda una referencia al mismo objeto Jugando

10 Niveles desde datos

Los niveles no se programan, se describen con datos: una cuadrícula de caracteres o un JSON donde cada símbolo es un tipo de casilla.

El juego lee el archivo y convierte cada símbolo en paredes, monedas, enemigos o puntos de inicio y salida.

Separar datos y código permite diseñar niveles sin tocar el programa, incluso con editores visuales como Tiled.

Ejemplo

```

1  import json
2
3  NIVEL = json.loads("""{"nombre": "Cueva 1", "mapa": [
4      "#####",
5      "#P..C...E#",
6      "#.####...#",
7      "#...C..C.#",
8      "#####"]}""")
9
10 TAM = 32
11 paredes, monedas, inicio, salida = [], [], None, None
12 for fila, linea in enumerate(NIVEL["mapa"]):
13     for col, celda in enumerate(linea):
14         pos = (col * TAM, fila * TAM)
15         if celda == "#":
16             paredes.append(pos)
17         elif celda == "C":
18             monedas.append(pos)
19         elif celda == "P":
20             inicio = pos
21         elif celda == "E":
22             salida = pos
23
24 print(NIVEL["nombre"], "|", len(paredes), "paredes |", len(monedas), "monedas")
25 print("Inicio:", inicio, "| Salida:", salida)
26 print("Tamaño:", len(NIVEL["mapa"][0]) * TAM, "x", len(NIVEL["mapa"]) * TAM, "px")

```

Salida real

```

Cueva 1 | 30 paredes | 3 monedas
Inicio: (32, 32) | Salida: (256, 32)
Tamaño: 320 x 160 px

```

Consejos

- Valida cada nivel al cargarlo: exactamente un inicio y al menos una salida.
- La librería pytmx lee mapas creados con el editor Tiled.

Ejercicio

Guarda tres niveles en archivos JSON y pasa al siguiente cuando el jugador llegue a la salida.

Práctica evaluable

Cargar y validar un nivel · La lógica de la lección, sin librerías externas

Escribe `cargar_nivel(mapa)`: `mapa` es una lista de strings donde `#` es pared, `C` moneda, `P` inicio del jugador, `E` salida y `.` vacío. Devuelve `{"paredes": cantidad, "monedas": [(col, fila), ...], "inicio": (col, fila), "salida": (col, fila)}`. Valida y lanza `ValueError` si las filas no miden lo mismo, si no hay exactamente un `P` o si no hay al menos un `E` (si hay varias salidas, devuelve la primera).

Plantilla para empezar

```
1 def cargar_nivel(mapa):  
2     """Interpreta y valida un mapa de texto."""
```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

¿Qué ventaja tiene describir niveles con datos?

- a) Evita usar clases
- b) Corre más rápido
- c) Se pueden crear y modificar niveles sin cambiar el código
- d) Ocupa menos disco

Misión del módulo

Agrega menú, pausa, game over y tres niveles cargados desde archivos a tu juego.

Módulo 6

Rendimiento y publicación

Requisitos: N4.5 Profiling y optimización, N3.10 Empaquetado y distribución. Aparece en el Nivel 4 del árbol.

11 Medir y optimizar el cuadro

A 60 FPS tienes unos 16 ms por cuadro para actualizar y dibujar todo. Si te pasas, el juego se traba.

`convert()` y `convert_alpha()` adaptan una imagen al formato de la pantalla y aceleran mucho el dibujo.

Mide antes de optimizar: `time.perf_counter` o `cProfile` te dicen qué parte del cuadro consume el tiempo.

Ejemplo

```
1 import pygame, timeit
2
3 pygame.init()
4 pantalla = pygame.display.set_mode((640, 360))
5 original = pygame.Surface((64, 64), depth=24)
6 convertida = original.convert()
7
8 t_original = timeit.timeit(lambda: pantalla.blit(original, (0, 0)), number=20_000)
9 t_convertida = timeit.timeit(lambda: pantalla.blit(convertida, (0, 0)),
10 ↪ number=20_000)
11 print(f"sin convert: {t_original:.3f}s | con convert: {t_convertida:.3f}s")
12 print(f"presupuesto por cuadro a 60 FPS: {1000 / 60:.1f} ms")
```

Salida real (los tiempos varían en cada equipo)

```
sin convert: 0.039s | con convert: 0.023s
presupuesto por cuadro a 60 FPS: 16.7 ms
```

Consejos

- Llama a `convert()` siempre después de crear la ventana; antes no conoce el formato de pantalla.
- Dibuja solo lo que está dentro de la cámara: lo que no se ve, no se dibuja.

Ejercicio

Muestra en pantalla los FPS reales con `reloj.get_fps()` y encuentra con `cProfile` la función más costosa de tu juego.

Práctica evaluable

Dibujar solo lo visible · La lógica de la lección, sin librerías externas

Para ahorrar tiempo de cuadro, solo se dibuja lo que está dentro de la cámara. Los objetos y la cámara son rectángulos (`x`, `y`, `ancho`, `alto`) en coordenadas del mundo.

- `visibles(objetos, camara)`: los objetos que se superponen con la cámara, en el mismo orden.
- `a_pantalla(objeto, camara)`: la posición (`x`, `y`) del objeto relativa a la esquina de la cámara.

Plantilla para empezar

```

1 def visibles(objetos, camara):
2     """Objetos dentro de la cámara."""
3
4
5 def a_pantalla(objeto, camara):
6     """(x, y) relativa a la cámara."""

```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

¿Cuánto tiempo hay por cuadro a 60 FPS?

- a) 60 ms
- b) 1 segundo
- c) 6 ms
- d) Unos 16,7 ms

12 Empaquetar y publicar

PyInstaller convierte tu juego en un ejecutable que se abre sin tener Python instalado.

Los recursos (imágenes, sonidos) deben empaquetarse junto al ejecutable y buscarse con una función que funcione tanto en desarrollo como empaquetado.

itch.io es la plataforma más amigable para publicar juegos independientes, gratis o pagos.

Ejemplo

```

1 import sys
2 from pathlib import Path
3
4 def recurso(ruta_relativa: str) -> Path:
5     """Encuentra assets en desarrollo y dentro del ejecutable de PyInstaller."""
6     base = Path(getattr(sys, "_MEIPASS", Path(__file__).parent))
7     return base / ruta_relativa
8
9 # imagen = pygame.image.load(recurso("assets/nave.png")).convert_alpha()
10
11 # En la terminal:
12 # pip install pyinstaller
13 # pyinstaller --onefile --windowed --add-data "assets:assets" juego.py

```

Resultado esperado

```

dist/juego.exe  <- tu juego listo para compartir
(en Windows usa --add-data "assets;assets")

```

Consejos

- pygbag publica juegos de pygame para jugarse directamente en el navegador.
- Prueba el ejecutable en una computadora sin Python antes de publicarlo.

Ejercicio

Empaqueta tu juego, súbelo a itch.io como borrador y pide a dos personas que lo prueben.

Práctica evaluable

Armar el comando de PyInstaller · La lógica de la lección, sin librerías externas

Escribe `comando_pyinstaller(script, sistema, assets="assets")`, que devuelve el comando para empaquetar el juego: `pyinstaller --onefile --windowed --add-data "assets;assets" juego.py`. En Windows el separador de `--add-data` es `;` y en el resto ("linux", "mac") es `:`. Si `assets` es `None`, el comando no lleva `--add-data`. Escribe también `nombre_ejecutable(script, sistema): "juego.exe" en Windows y "juego" en el resto.`

Plantilla para empezar

```
1 def comando_pyinstaller(script, sistema, assets="assets"):
2     """comando_pyinstaller("juego.py", "windows")"""
3
4
5 def nombre_ejecutable(script, sistema):
6     """juego.py -> "juego.exe" en Windows"""
```

Escríbela y evalúala en la app (o en tu IDE). Las pistas y la solución están al final del manual. Al aprobarla, anótala en la **Bitácora de prácticas** del Cuadernillo.

Autoevaluación

¿Para qué sirve `sys._MEIPASS`?

- a) Indica la carpeta temporal donde PyInstaller descomprime los recursos
- b) Mide FPS
- c) Activa el audio
- d) Guarda el récord

Misión del módulo

Optimiza tu juego para mantener 60 FPS y publícalo como ejecutable en itch.io.

Proyecto integrador

Plataformas 2D con tres niveles

Un juego de plataformas completo con menú, tres niveles diseñados con datos, enemigos, monedas, sonido, récord guardado y ejecutable publicado.

Paso a paso

1. Estructura el juego con estados (menú, jugando, pausa, game over).
2. Crea jugador, enemigos y monedas como sprites con animaciones.
3. Implementa física de salto y colisiones por ejes.
4. Carga tres niveles desde JSON y avanza al llegar a la salida.
5. Agrega música, efectos y récord en un archivo.
6. Optimiza a 60 FPS estables y publica en itch.io.

Criterios de éxito

Funciona sin errores con entradas normales y extrañas, está dividido en funciones o clases con nombres claros, tiene docstrings y tests donde corresponde, y registraste el proceso en la Plantilla de proyecto del Cuadernillo.

Checklist de cierre

- ☐ Misión cumplida: El bucle de juego
- ☐ Misión cumplida: Sprites y animación
- ☐ Misión cumplida: Colisiones
- ☐ Misión cumplida: Física, sonido y eventos
- ☐ Misión cumplida: Estados y niveles
- ☐ Misión cumplida: Rendimiento y publicación
- ☐ Terminé el proyecto "Plataformas 2D con tres niveles"

Solucionario de autoevaluación

- 1. La ventana y el bucle:** c) Limita el juego a 60 cuadros por segundo. Sin límite, el juego correría a velocidades distintas en cada computadora.
- 2. Dibujar y mover con delta time:** b) Para que el movimiento no dependa de los FPS. dt convierte 'píxeles por segundo' en 'píxeles por este cuadro'.
- 3. Sprites y grupos:** c) image y rect. draw() dibuja cada image en la posición de su rect.
- 4. Animación por cuadros:** a) La animación mantiene su velocidad aunque cambien los FPS. Con frames, la animación se aceleraría o frenaría según el equipo.
- 5. Detectar choques:** b) Quita del grupo los sprites que chocaron. Ideal para monedas o balas que desaparecen al tocar.
- 6. Responder a colisiones:** c) Para saber de qué lado vino el choque y corregir sin engancharse. Separar X e Y deja claro si el choque fue lateral o vertical.
- 7. Gravedad y salto:** a) Caídas cada vez más rápidas que atraviesan el suelo. Limitar vy mantiene la caída controlada y las colisiones confiables.
- 8. Eventos propios, oleadas y sonido:** a) Envía EVENTO a la cola cada 100 ms. Lo recibes en el bucle de eventos como cualquier otro.
- 9. Máquina de estados:** d) Pausa guarda una referencia al mismo objeto Jugando. Pausa devuelve self.juego, la misma instancia que se pausó.
- 10. Niveles desde datos:** c) Se pueden crear y modificar niveles sin cambiar el código. Datos y lógica separados: diseñar niveles deja de ser programar.
- 11. Medir y optimizar el cuadro:** d) Unos 16,7 ms. $1000 \text{ ms} / 60 \text{ cuadros} \approx 16,7 \text{ ms}$.
- 12. Empaquetar y publicar:** a) Indica la carpeta temporal donde PyInstaller descomprime los recursos. Solo existe dentro del ejecutable; en desarrollo se usa la carpeta del archivo.

Pistas y soluciones de las prácticas

Usa las pistas de a una, y solo después de intentarlo. Hay más de una solución correcta: si la tuya pasa las pruebas, compárala con esta para aprender otra forma de pensarlo.

1. La ventana y el bucle · Contador de FPS y presupuesto por cuadro

Pista 1: Promedio = suma / cantidad.

Pista 2: Revisa la lista vacía antes de dividir.

```

1 def calcular_fps(duraciones_ms):
2     """FPS promedio."""
3     if not duraciones_ms:
4         return 0
5     promedio = sum(duraciones_ms) / len(duraciones_ms)
6     return round(1000 / promedio)
7
8
9 def presupuesto(fps):
10    """Milisegundos por cuadro."""
11    return round(1000 / fps, 1)

```

Por qué funciona: `reloj.get_fps()` hace esta cuenta por ti. Entenderla te ayuda a leer el presupuesto: a 60 FPS, todo el cuadro tiene menos de 17 ms.

2. Dibujar y mover con delta time · Mover y rebotar

Pista 1: Reubicar evita que la pelota quede pegada fuera del borde.

Pista 2: `abs()` fija la dirección sin importar cómo venía.

```

1 def mover_y_rebotar(x, vx, radio, ancho, dt):
2     """Devuelve (x, vx) tras moverse dt segundos."""
3     x += vx * dt
4     if x - radio < 0:
5         x, vx = radio, abs(vx)
6     elif x + radio > ancho:
7         x, vx = ancho - radio, -abs(vx)
8     return x, vx

```

Por qué funciona: Usar `abs` en vez de invertir el signo evita un error clásico: si la pelota quedó fuera dos cuadros seguidos, invertir dos veces la devolvería hacia la pared.

3. Sprites y grupos · Una nave que no se sale de la pantalla

Pista 1: Guarda todos los parámetros en `self`.

Pista 2: `max(0, min(valor, maximo))` limita un valor a un rango.

```

1 class Nave:
2     def __init__(self, x, ancho_pantalla=640, ancho=40, velocidad=5):
3         self.x = x
4         self.ancho_pantalla = ancho_pantalla
5         self.ancho = ancho
6         self.velocidad = velocidad
7
8     def mover(self, direccion):
9         """-1 izquierda, 0 quieta, 1 derecha. Devuelve la nueva x."""
10        nuevo = self.x + direccion * self.velocidad
11        self.x = max(0, min(nuevo, self.ancho_pantalla - self.ancho))
12        return self.x

```

Por qué funciona: `max(0, min(...))` es el equivalente manual de `rect.clamp_ip`: limita la posición en una línea, sin `if` para cada borde.

4. Animación por cuadros · Elegir el cuadro de una animación

Pista 1: `int(tiempo * fps)` cuenta los cuadros transcurridos.

Pista 2: Para no repetir: `min(pasados, cantidad - 1)`.

```

1 def indice_cuadro(tiempo, fps, cantidad, bucle=True):
2     """Índice del cuadro a mostrar."""
3     pasados = int(tiempo * fps)
4     if bucle:
5         return pasados % cantidad
6     return min(pasados, cantidad - 1)

```

Por qué funciona: Calcular el cuadro a partir del tiempo (y no del número de frames del juego) mantiene la velocidad de la animación aunque el juego vaya a 30 o a 144 FPS.

5. Detectar choques · Colisiones de rectángulos

Pista 1: Dos rectángulos chocan si se superponen en `X ** y **` en Y.

Pista 2: En X: el inicio de uno está antes del final del otro, y al revés.

```

1 def colisionan(a, b):
2     """True si los rectángulos se superponen."""
3     ax, ay, aw, ah = a
4     bx, by, bw, bh = b
5     return ax < bx + bw and ax + aw > bx and ay < by + bh and ay + ah > by
6
7
8 def recoger(jugador, monedas):
9     """-> (restantes, recogidas)"""
10    restantes = [m for m in monedas if not colisionan(jugador, m)]
11    return restantes, len(monedas) - len(restantes)

```

Por qué funciona: Esta prueba de cuatro comparaciones (AABB) es exactamente lo que hace `Rect.colliderect`. `recoger` es `spritecollide` con `dokill=True`, escrito a mano.

6. Responder a colisiones · Aterrizar sobre plataformas

Pista 1: Reutiliza la idea de `colisionan` con la nueva `y`.

Pista 2: La dirección de `vy` dice de qué lado vino el choque.

```

1 def mover_vertical(jugador, vy, plataformas):
2     """-> (y, vy, en_suelo)"""
3     x, y, w, h = jugador
4     y += vy
5     en_suelo = False
6     for px, py, pw, ph in plataformas:
7         if x < px + pw and x + w > px and y < py + ph and y + h > py:
8             if vy > 0:
9                 y, vy, en_suelo = py - h, 0, True
10            elif vy < 0:
11                y, vy = py + ph, 0
12    return y, vy, en_suelo

```

Por qué funciona: Resolver solo el eje Y (después de haber resuelto X) es lo que evita engancharse en las esquinas: la dirección de la velocidad indica sin ambigüedad si el jugador aterrizó o chocó con la cabeza.

7. Gravedad y salto · Simular un salto

Pista 1: En pantalla, Y crece hacia abajo: el suelo es 0 y arriba es negativo.

Pista 2: `min(y + vy, 0)` impide atravesar el suelo.

```

1 def simular_salto(gravedad, fuerza, vel_max, frames):
2     """Alturas del jugador en cada cuadro."""
3     y, vy = 0, -fuerza # y crece hacia abajo, como en pantalla
4     alturas = []
5     for _ in range(frames):
6         vy = min(vy + gravedad, vel_max)
7         y = min(y + vy, 0)
8         if y == 0:
9             vy = 0
10        alturas.append(-y)
11    return alturas

```

Por qué funciona: El orden importa: primero la gravedad cambia la velocidad y después la velocidad cambia la posición. Con números enteros la simulación es exacta y fácil de seguir en papel.

8. Eventos propios, oleadas y sonido · Generador de oleadas

Pista 1: `while True` con `yield` es seguro: el generador se pausa en cada valor.

Pista 2: `[next(g) for _ in range(n)]`

```

1 def oleadas(base=3, incremento=2):
2     """Generador infinito de oleadas."""
3     n = 1
4     while True:
5         yield {"numero": n, "enemigos": base + incremento * n, "velocidad": 1 +
6             ↪ 0.25 * n}
7         n += 1
8
9 def tomar(generator, n):
10     """Los próximos n valores del generador."""
11     return [next(generator) for _ in range(n)]

```

Por qué funciona: Un generador infinito describe la regla de dificultad sin calcular oleadas de más. El juego pide la siguiente solo cuando la necesita.

9. Máquina de estados · Máquina de estados con una tabla

Pista 1: Las tuplas pueden ser claves de diccionario.

Pista 2: El estado actual siempre es el último de la lista.

```

1 def transicion(estado, evento):
2     """Nuevo estado según la tabla."""
3     return TRANSICIONES.get((estado, evento), estado)
4
5
6 def recorrer(eventos, inicial="menu"):
7     """Estados visitados."""
8     estados = [inicial]
9     for evento in eventos:
10         estados.append(transicion(estados[-1], evento))
11     return estados

```

Por qué funciona: La tabla y las clases del patrón State son dos formas del mismo concepto. La tabla brilla cuando los estados no tienen lógica propia; las clases, cuando cada estado tiene comportamiento.

10. Niveles desde datos · Cargar y validar un nivel

Pista 1: `{len(fila) for fila in mapa}` tiene un solo elemento si todas las filas miden lo mismo.

Pista 2: Recorre con dos `enumerate` anidados: fila y columna.

```

1 def cargar_nivel(mapa):
2     """Interpreta y valida un mapa de texto."""
3     if len({len(fila) for fila in mapa}) > 1:
4         raise ValueError("Todas las filas deben medir lo mismo")
5     paredes, monedas, inicios, salidas = 0, [], [], []
6     for fila, linea in enumerate(mapa):
7         for col, celda in enumerate(linea):
8             if celda == "#":
9                 paredes += 1
10            elif celda == "C":
11                monedas.append((col, fila))
12            elif celda == "P":
13                inicios.append((col, fila))
14            elif celda == "E":
15                salidas.append((col, fila))
16        if len(inicios) != 1:
17            raise ValueError(f"Debe haber exactamente un inicio y hay {len(inicios)}")
18        if not salidas:
19            raise ValueError("El nivel necesita al menos una salida")
20    return {"paredes": paredes, "monedas": monedas, "inicio": inicios[0], "salida":
    ↪ salidas[0]}

```

Por qué funciona: Validar al cargar convierte un nivel mal diseñado en un mensaje claro, en lugar de un juego que se rompe a mitad de partida.

11. Medir y optimizar el cuadro · Dibujar solo lo visible

Pista 1: Ver si un objeto es visible es una colisión con el rectángulo de la cámara.

Pista 2: Restar la posición de la cámara pasa de coordenadas del mundo a la pantalla.

```

1 def _chocan(a, b):
2     ax, ay, aw, ah = a
3     bx, by, bw, bh = b
4     return ax < bx + bw and ax + aw > bx and ay < by + bh and ay + ah > by
5
6
7 def visibles(objetos, camara):
8     """Objetos dentro de la cámara."""
9     return [o for o in objetos if _chocan(o, camara)]
10
11
12 def a_pantalla(objeto, camara):
13     """(x, y) relativa a la cámara."""
14     return objeto[0] - camara[0], objeto[1] - camara[1]

```

Por qué funciona: Descartar lo que no se ve (culling) es una de las optimizaciones más rentables: en un nivel grande, se dibuja una fracción pequeña de los objetos.

12. Empaquetar y publicar · Armar el comando de PyInstaller

Pista 1: Arma una lista de partes y únelas con " ".join.

Pista 2: El separador cambia solo según el sistema.

```
1 def comando_pyinstaller(script, sistema, assets="assets"):
2     """Comando de PyInstaller según el sistema."""
3     partes = ["pyinstaller", "--onefile", "--windowed"]
4     if assets:
5         sep = ";" if sistema == "windows" else ":"
6         partes.append(f'--add-data "{assets}{sep}{assets}"')
7     partes.append(script)
8     return " ".join(partes)
9
10
11 def nombre_ejecutable(script, sistema):
12     """Nombre del archivo final."""
13     base = script.rpartition(".")[0] or script
14     return base + ".exe" if sistema == "windows" else base
```

Por qué funciona: Generar el comando con código evita el error más común al publicar: usar el separador de otro sistema operativo y que el ejecutable arranque sin sus imágenes.